C++ Support in TRENTOS

Current status:

- **no** c++ library is being linked
  - Possible options: https://libcxx.llvm.org/
  - new / delete need to be implemented as wrappers around malloc/free from musllibc
  - cout not implemented / should be a wrapper over printf
- STL support implemented using the Embedded Template Library https://www.etlcpp.com/
- example can be found here https://bitbucket.hensoldt-cyber.systems/projects/SS/repos/demo_hello_world/browse?at=refs%2Fheads%2Ftest-cpp
- used in RPi4 circle library

Build system

The sel4 cmake build system was extended with the addition of **CXX_FLAGS** as an option for **DeclareCAmkESComponent**. These flags are automatically passed to the compilation of source files having the CPP file extension

Limitations

- The following compilation flags are needed / the following functionality needs to be disabled
  - -fno-exceptions
    - exceptions not normally used in embedded systems
    - requires the implementation of __cxa_end_cleanup / __gxx_personality_v0
  - -fno-rtti
    - rtti not normally used in embedded systems
    - requires the implementation of __si_class_type_info
  - -fno-threadsafe-statics
    - requires a mutex from the environment, currently not implemented
    - even if we don't implement multi-threaded components, the existence of interface threads might force us to implement this
- operator new, new[], delete and delete[] need to be implemented

C++ operators implementation

```
void*
operator new(size_t n)
{
    void* const p = malloc(n);

    return p;
}

void*
operator new[](size_t n)
{
    void* const p = malloc(n);

    return p;
}

void
operator delete(void* p, size_t sz)
{
          free(p);
}

void
operator delete[](void* p)
{
    free(p);
}
```

How to use

To declare a camkes c++ component, the project type needs to be changes to CXX. Optionally the CXX_FLAGS can also be used.

Mixing of C and C++ code is possible in a component, but "extern" needs to be used correctly.

Declaring a C++ component

```
project(tests_hello_world CXX)

DeclareCAmkESComponent(
    cpp_test
    INCLUDES
        # no include paths needed
    SOURCES
        main.cpp
                source.c
    C_FLAGS
        -Wall
        -Werror
    CXX_FLAGS
        -Wall
        -Werror
        -fno-exceptions
        -fno-rtti
    LIBS
        os_core_api
)

DeclareAndCreateCamkESSystem(demo_hello_world.camkes)
```

The camkes callback functions (run / pre_init / post_init & co) can be implemented in a C++ file as long as they're declared as extern.

cmakes callback functions

```
extern "C" int
run()
{
    printf("Hello world!");

    return 0;
}
```

The following features are demonstrated in the test app:

- templates
- OOP (derivating classes, vtables)
- lambdas

Templates, OOP, lambdas example

```
template<class T>
class temp
{
    T var;

  public:
    template<class Y>
    temp(Y tmp)
    {
        var = tmp;
        printf("Constructor is %d\n", var);
    }
};

class Runner
{
  public:
    Runner() { printf("Constructor is called\n"); }
    virtual ~Runner() { printf("Destructor is called\n"); }

    void run() { printf("hello world!\n"); }
};

class Derived : public Runner
{
  public:
    Derived() { printf("Derived Constructor is called\n"); }
    ~Derived() { printf("Derived Destructor is called\n"); }

    void run() { printf("hello world!\n"); }
};

extern "C" int
run()
{
    temp<int> a(256);
    temp<char> b(256);
    Runner    r, *s;

    s = new Runner();
    s->run();
    delete s;

    s = new Runner();
    s->run();
    delete s;

    r.run();

    auto p = new Runner();
    p->run();

    s = new Derived();

    delete s;
    delete p;

    int m = 0;
    int n = 0;
    [&, n](int a) mutable { m = ++n + a; }(4);

    printf("%d %d \n",m,n);

    return 0;
}
```

Embedded Template Library

As a STL implementation, the Embedded Template Library (https://www.etlcpp.com/) can be used, allowing the creation of containers with predefined maximum storage :

ETL code example

```
etl::queue<int, 5> q;
for (int i = 0; i < 10; i++)
{
    if (!q.full())
        q.push(i);
}

printf("Queue size %d\n", q.size());
for (int i = 0; i < 10; i++)
{
    if(!q.empty())
    {
        printf("Got element %d\n",q.front());
        q.pop();
    }
}
```

Currently, the ETL is added as a folder in the target component (demo_hello_world, test-cpp branch), from which a library is built.

ETL library

```
project(etl CXX)

add_library(${PROJECT_NAME} INTERFACE)

target_sources(${PROJECT_NAME}
INTERFACE
)

target_include_directories(${PROJECT_NAME}
INTERFACE
etl
)
```

(Possible) Future Steps

- create a ETL library in sandbox
- create a trentos_cpp library which implmenets
    - new / delete operators
    - mutex needed for thread-safe statics